

Week 10– Hardware integration

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



WE ARE

HUMBER

Review from Last Week



- Gazebo Sim (Harmonic) provides realistic, physics-based environments for testing robots safely.
- Integrated with ROS 2 Jazzy using the `ros_gz_sim` bridge for full sensor and actuator interaction.
- Use **launch files** to automate startup: `robot_state_publisher`, `ros_gz_sim`, and robot spawner.
- Extend URDF with `<gazebo>` tags and **plugins** for control, sensors, and visuals.
- Always enable `use_sim_time` for synchronized simulation.
- Customize visuals with Gazebo materials (e.g., `Gazebo/Blue`).
- Add sensors like LIDAR or cameras using `<sensor>` and plugin tags.
- **Key takeaway:** Gazebo + ROS 2 enable full virtual prototyping—build, test, and control robots in simulation before deploying to hardware.

Agenda for today's training session



Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
- Integrate hardware components (sensors and actuators) in a ROS2 system.

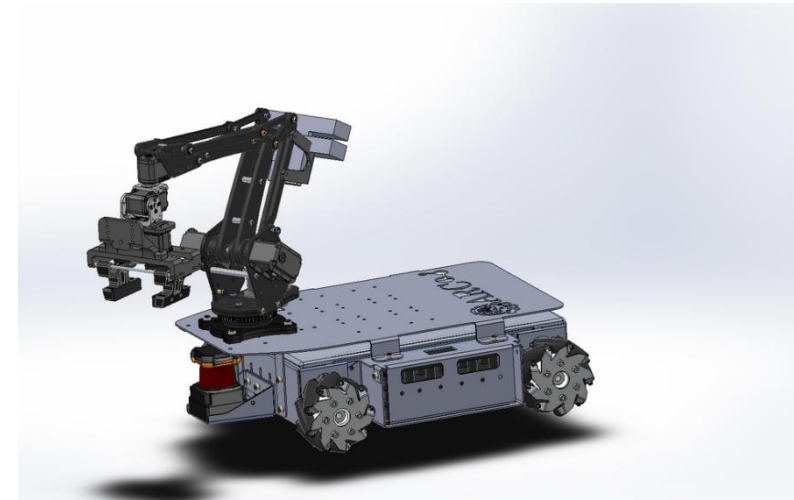
Problem we will be solving today:

- Electrical analysis
- Sensors
- Actuators

Robotic Design



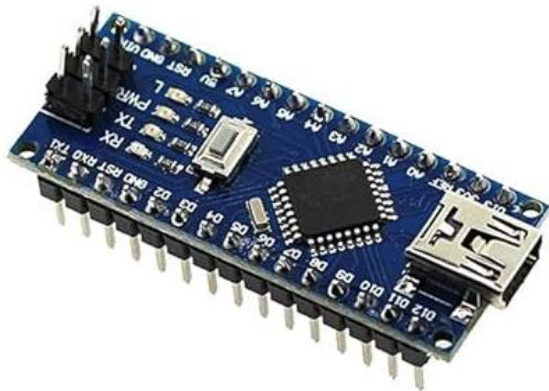
- **Selecting operating voltages**
- **Understanding current requirements**
- **Power regulation & distribution**
- **Battery selection & safety**
- **Wiring, connectors, and protection**
- **Sensor Integration: Lidar sensor**
- **Actuator integration**
- **Serial communication and networking**
- **Encoders**



[source](#)

Electrical Analysis

- Robot requires two voltage levels
 - 12 V – motors and main supply
 - 5 V – logic components (Pi, Arduino, sensors)
- Some subsystems may use 3.3 V (future expansion)



Electrical Analysis

$$P = V \times I$$

Where:

- P = power (Watts)
- V = voltage (Volts)
- I = current (Amperes)



Example:

$5\text{ V} \times 5\text{ A} = 25\text{ W}$ total power for electronics.

Current from Input Voltage (Power Conservation)

If regulator efficiency = η :

$$I_{in} = \frac{P_{out}}{V_{in} \cdot \eta}$$

For ideal case ($\eta = 1$):

$$I_{in} = \frac{P_{out}}{V_{in}}$$

 Example:

$$I_{in} = \frac{25 \text{ W}}{12 \text{ V}} = 2.08 \text{ A}$$

$\approx 2.5 \text{ A}$ allowing for efficiency losses.

Regulator Efficiency

$$\eta = \frac{P_{out}}{P_{in}} \times 100\%$$

Typical values:

- Linear regulator $\approx 60\text{--}70\%$
- Switching regulator (buck) $\approx 85\text{--}95\%$



Example:

If $5\text{ V} \times 5\text{ A} = 25\text{ W}$ output and $\eta = 90\%$,

$$P_{in} = \frac{25}{0.9} = 27.8\text{ W}$$

so $I_{in} = 27.8/12 = 2.32\text{ A}$.

Parallel Loads (Total Current)

$$I_{total} = \sum I_{device}$$



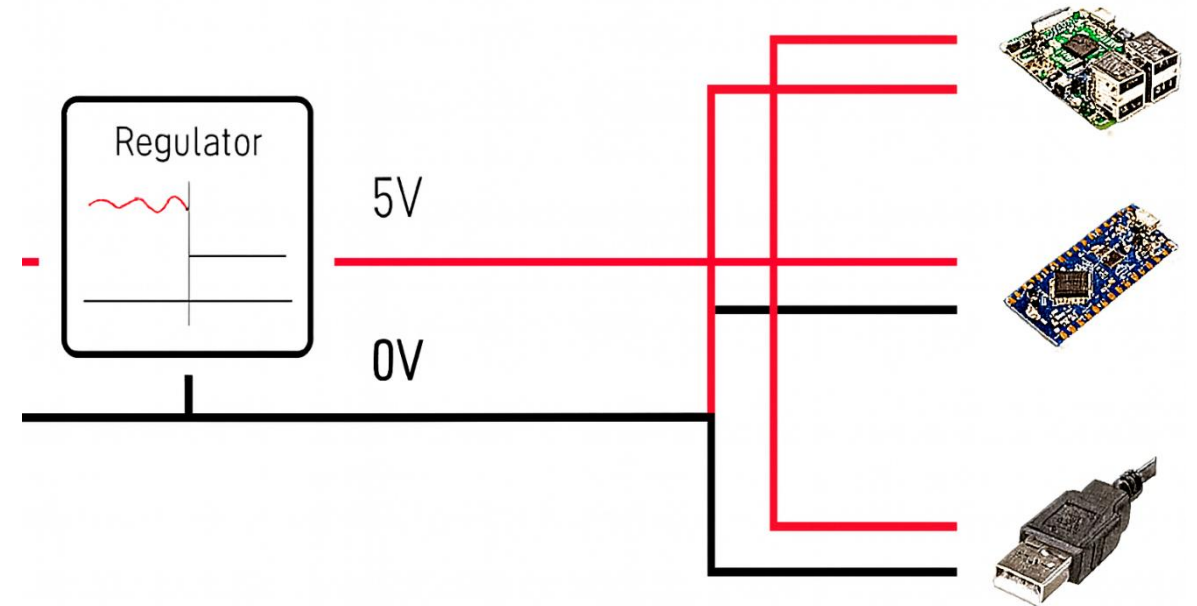
Example:

Pi (1.5 A) + Arduino (0.1 A) + LiDAR (1 A) + Display (1 A) ... $\approx$ 5 A total.

Voltage Regulators

Two main types:

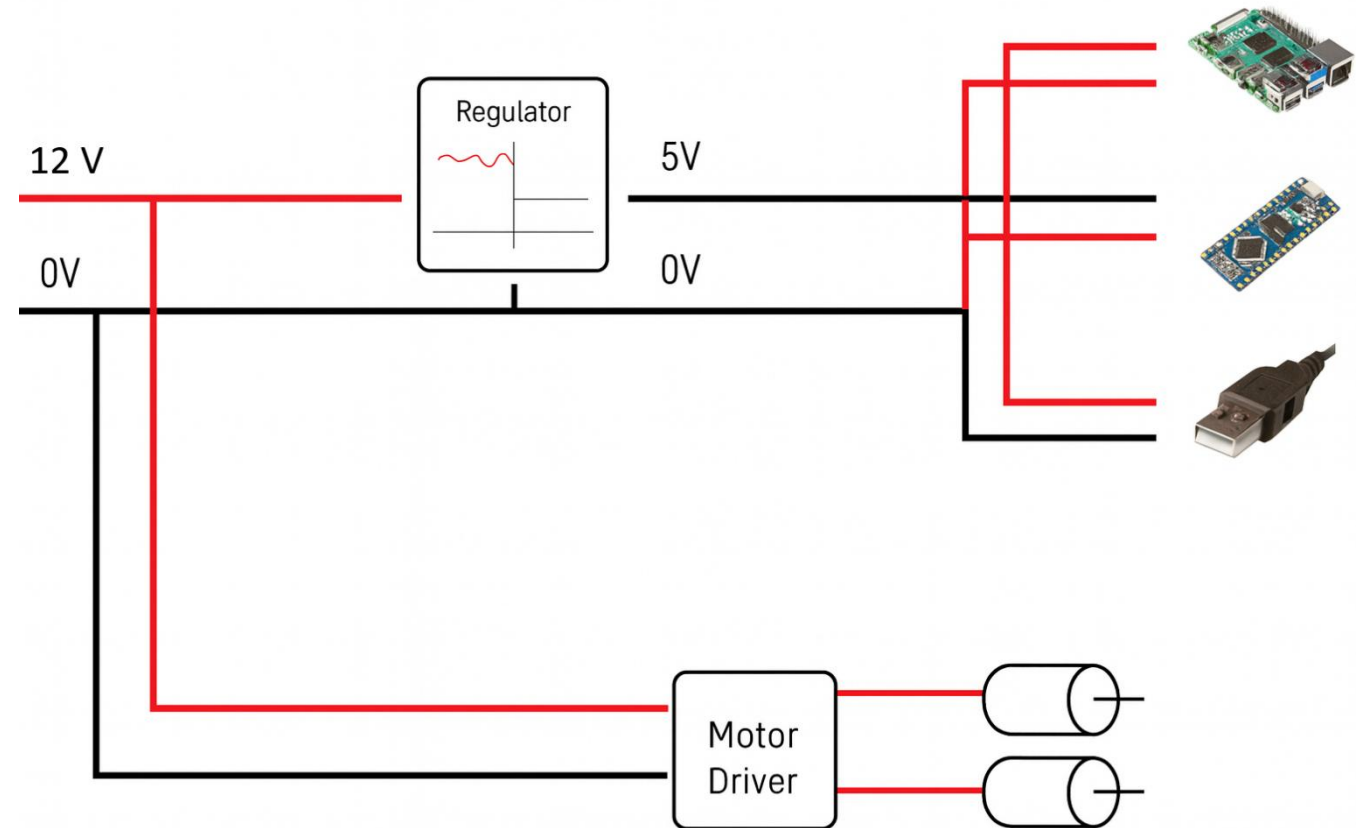
- **Linear Regulators**
 - Simple, clean output, cheap
 - Inefficient at high current
- **Switching Regulators**
 - High efficiency
 - Ideal for robotics projects



Supply Voltage

- **Chosen to match motor rating (typically 12 V)**
- **DC motors selection. Examples: 6 V, 12 V, 24 V motors**

 **Summary:**
12 V supply → Regulated 5 V for electronics



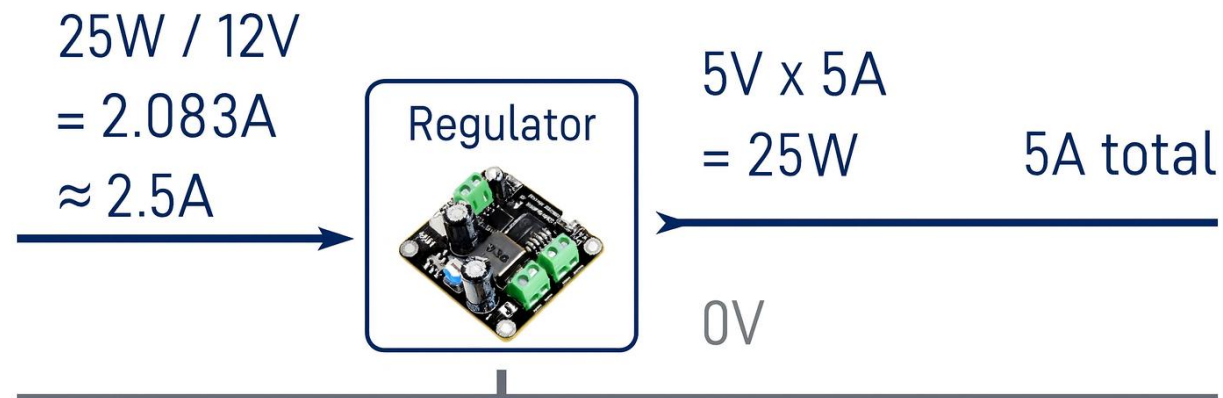
Current Considerations

- Example 5 V Components**

Device	Typical Current	Notes
Raspberry Pi	1.5 A	Max ≈ 7 W
Arduino	0.1 A	Board + pins
RPLidar A1	1 A	Start ≈ 600 mA
Pi Camera	0.1 A	≈ 100 mA extra
Depth Camera	0.9 A	~ 4.5 W
Motor Driver Logic	0.1 A	Control only
LCD Screen	1 A	Display

Regulator Sizing

- Regulator must handle $\geq 5\text{ A}$ output
- Input $12\text{ V} \rightarrow$ Output $5\text{ V @ } 25\text{ W}$
- Draws $\approx 2.1\text{--}2.5\text{ A}$ from 12 V supply
- Choose reliable 25 W converter



Motor Current

- Depends on load & torque
- Check **datasheet** for stall & rated current
 - Example: 1.8 A (stall), 0.75 A (rated)
- Two motors → ≈ 3.6 A total

⚙ Total System Current ≈ 6 A @ 12 V

Voltage	No-load Speed	Gear Ratio	No-load Current (A)	No-load Speed (rpm)	Rated Torque (kg·cm)	Rated Speed (rpm)	Rated Current (A)	Stall Torque (kg·cm)	Stall Current (A)
DC 12.0 V	110 rpm	1 : 75	≤ 0.15	$110 \pm 10\%$	8.0	$65 \pm 10\%$	≤ 0.75	≤ 9.0	1.8
DC 12.0 V	190 rpm	1 : 45	≤ 0.15	$190 \pm 10\%$	5.0	$100 \pm 10\%$	≤ 0.75	≤ 9.0	1.8

Power Source (LiPo Safety)

- **Requirements:**
- **~ 12 V nominal**
- **≥ 6 A continuous**
-  **Best Option: 3-Cell LiPo (11.1 V nominal)**
- **3S = 3 cells $\times$ 3.7 V = 11.1 V**
-  **Handle with Care**
- **Never short or over-discharge**
- **Charge with proper LiPo charger**
- **Store in fireproof container**
- **Supervise charging**

Wiring & Connectors

- Choose wire gauge for expected current
 - Reference AWG vs Current table
- Use connectors rated above max current
 - e.g. XT60 (60 A), RCY (3 A)
- Avoid undersized wires → overheating



Power Switch & Fuse Protection

- **Power Switch**
 - **Safe way to disconnect battery**
 - **Prevents shorts while plugging in**
 - **Mount externally for easy access**
- **Fuse**
 - **Purpose: Stop short circuits & prevent fires**
 - **Fuse opens when current > rated limit**
 - **Protects battery and wiring (not components)**
-  **Place fuse close to battery terminal**

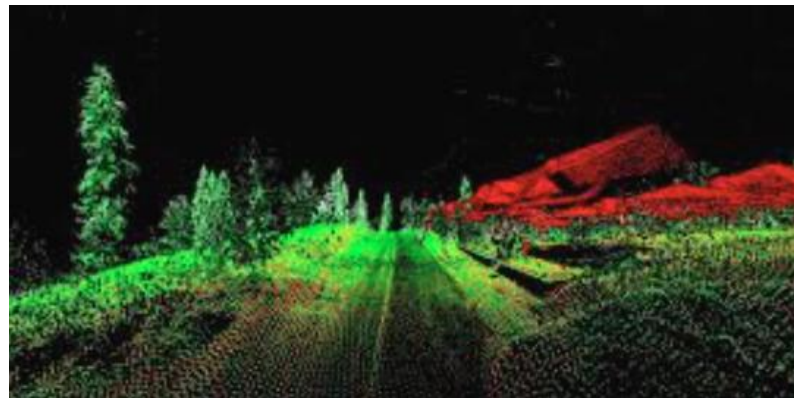
Lidar sensor

- Light Detection And Ranging
- Similar to sonar (sound) and radar (radio)
- Uses **laser light** to measure distance
- Provides range data to sense surroundings
- Builds a **local map** for navigation and obstacle detection



LiDAR Types

Type	Description	Typical Use
1D	Measures single distance point	UAV height or wall distance
2D	Sweep in one plane	Mobile robot mapping
3D	Captures 3D space	Autonomous vehicles, 3D imaging



Basic LiDAR Measurement Principle

Concept: Distance is measured using the **time-of-flight (ToF)** of a laser pulse.

Equation (Slide Example):

$$d = \frac{c \cdot t}{2}$$

Where:

- d = distance to object
- c = speed of light (3×10^8 m/s)
- t = time between emission and reflection
- LiDAR sends a light pulse and measures time until reflection.
- The factor of 2 accounts for the round trip.
- ROS drivers internally convert this to range data in **LaserScan** messages.

Scan Resolution & Sampling

Concept: The resolution determines how many rays per revolution.

Equation:

$$\Delta\theta = \frac{\text{FOV}}{N}$$

Where:

- $\Delta\theta$ = angular resolution (rad)
- **FOV** = Field of View (e.g., $360^\circ = 2\pi$)
- N = number of laser samples per revolution

Example:

For 360 samples per revolution:

$$\Delta\theta = \frac{2\pi}{360} = 1^\circ$$

LiDAR Point Density (2D Coverage)

Concept: How many points per meter can be captured at a given range.

Equation:

$$\rho = \frac{N}{2\pi r}$$

Where:

- ρ = point density (points/m)
- N = number of samples per revolution
- r = range to target

Why 2D LiDAR?

- Most common and affordable
- Great for **floor mapping** and navigation
- Wide variety of models (RPLIDAR, Hokuyo, SICK, etc.)
- Supported in ROS via standardized drivers

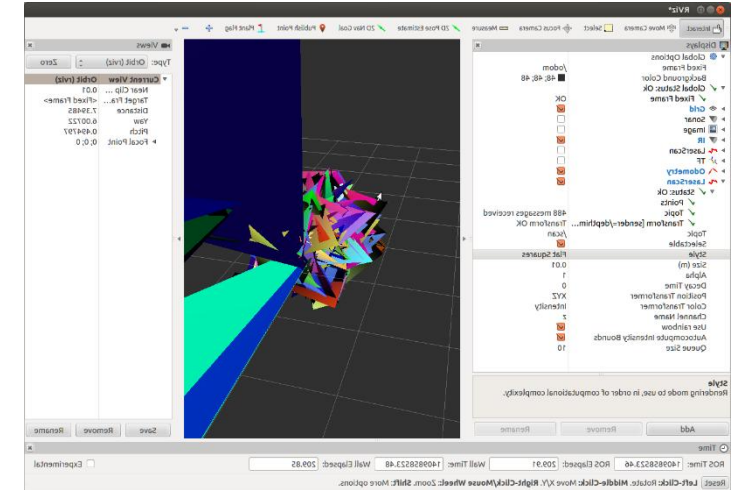
LiDAR in ROS

- ROS has strong LiDAR integration
- Drivers handle **device-specific communication**
- ROS publishes standardized topics → easy integration
- Works with simulation (Gazebo) and hardware

2D LiDAR Message Type

sensor_msgs/LaserScan

- Represents one full 2D sweep
- Contains range data (in meters)
- Includes frame reference (TF frame)
- Used for navigation, mapping, and obstacle avoidance



[source](#)

3D LiDAR Message Type

sensor_msgs/PointCloud2

- Used for 3D LiDAR and depth cameras
- Each point = (x, y, z, intensity)
- Provides rich environmental perception
- Used for SLAM, 3D mapping, and object recognition



[source](#)

Focus of This Lecture

- We'll focus on **2D LiDAR** integration
- All setup steps apply to **simulation and real sensors**

Simulating LiDAR in Gazebo

- Add LiDAR sensor to your robot model
- Define new link and joint in URDF/Xacro
- Attach the sensor to chassis
- Use a **fixed joint** for LiDAR frame



LiDAR Link in URDF (Example)

```
<joint name="laser_joint" type="fixed">
  <parent link="chassis"/>
  <child link="laser_frame"/>
  <origin xyz="0.1 0 0.175" rpy="0 0 0"/>
</joint>

<link name="laser_frame">
  <visual>
    <geometry><cylinder radius="0.05" length="0.04"/></geometry>
    <material name="red"/>
  </visual>
</link>
```

Gazebo Material

```
<gazebo reference="laser_frame">  
  <material>Gazebo/Red</material>  
</gazebo>
```

- Adds visible color for simulation
- Used only by Gazebo

Adding a Sensor Tag

Attach to the `<gazebo>` block:

xml

```
<sensor name="laser" type="ray">  
  <visualize>true</visualize>  
  <update_rate>10</update_rate>  
</sensor>
```

- Creates **virtual LiDAR** in simulation
- 10 Hz update rate

Ray Sensor Parameters

```
<ray>
  <scan>
    <horizontal>
      <samples>360</samples>
      <min_angle>-3.14</min_angle>
      <max_angle>3.14</max_angle>
    </horizontal>
  </scan>
  <range>
    <min>0.3</min>
    <max>12</max>
  </range>
</ray>
```

- 360° scan
- 12 m max range

ROS Plugin Setup

```
<plugin name="laser_controller" filename="libgazebo_ros_ray_sensor.so">
  <ros>
    <argument>~/out:=scan</argument>
  </ros>
  <output_type>sensor_msgs/LaserScan</output_type>
  <frame_name>laser_frame</frame_name>
</plugin>
```

- Publishes `/scan` topic
- Links sensor to `laser_frame`

Visualizing in RViz

- Launch Gazebo + robot state publisher
- In RViz:
 - Add → *LaserScan*
 - Topic: `/scan`
 - Fixed Frame: `odom`
- Observe 2D scan data in real time

Real LiDAR Setup

- **PLIDAR C1 (SLAMTEC) or similar**
 - Affordable 2D LiDAR
 - Works via USB interface
 - Supported by ROS package `rplidar_ros`



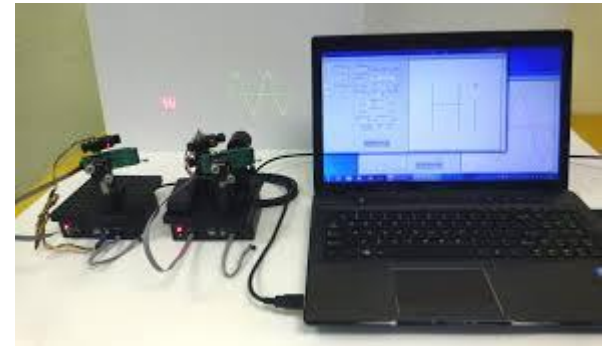
Connecting the Device

- Plug in via **micro-USB** cable
- Try multiple cables if not detected
- Verify connection:

```
bash
```

```
ls /dev/ttyUSB*
```

- Adapter connects to LiDAR and PC



Install ROS Driver

```
bash
```

```
sudo apt install ros-jazzy-rplidar-ros
```

(replace `foxy` → `jazzy` if needed)

- Installs RPLIDAR ROS driver
- Automatically publishes **LaserScan** topic

Running the Driver

```
bash
```

```
ros2 run rplidar_ros rplidar_composition --ros-args \  
  -p serial_port:=/dev/ttyUSB0 \  
  -p frame_id:=lidar_link \  
  -p angle_compensate:=true \  
  -p scan_mode:=Standard
```

- Runs LiDAR node
- Publishes `/scan` topic

Viewing the Data

In RViz:

- Fixed Frame → `laser_frame`
- Add → *LaserScan*
- Topic → `/scan`
- Observe real LiDAR scan in real-time

Extra Tips

- **Stable Serial Path:**

Use `/dev/serial/by-id` or `/dev/serial/by-path`

- **Control Motor:**

```
bash
```

```
ros2 service call /stop_motor std_msgs/Empty {}  
ros2 service call /start_motor std_msgs/Empty {}
```

- **Kill Node:**

```
bash
```

```
killall rplidar_composition
```

Adding to Launch File

- Automate the driver start-up
- Add parameters and frame config
- Ensures consistent setup for lab exercises

LIDAR Summary

- LiDAR = key for robot perception
- ROS supports both **2D (LaserScan)** and **3D (PointCloud2)**
- Simulation setup in Gazebo
- Real-world testing with RPLIDAR A1
- Visualization through RViz

Actuator Integration: Motors & Control Systems

- Convert **electrical energy** → **mechanical motion**
- Enable movement of **wheels, arms, grippers**
- Core of every mobile robot system

$$P_{mech} = \tau \cdot \omega$$

Type	Pros	Use
DC Brushed	Simple	Small mobile robots
DC Brushless	Efficient	Drones / fans
Stepper	High precision	Positioning
Servo	Angle control	Robotic arms

DC Motors

- Inexpensive and easy to drive
- Work directly with 12 V supply → ideal for our robot
- Controlled via **PWM + H-Bridge**

$$V = IR + L \frac{dI}{dt} + k_e \omega$$

- V : applied voltage I : current R : resistance
- L : inductance $k_e \omega$: back-EMF
- Back-EMF opposes input voltage → limits speed

Torque Generation, Speed, and Power

$$\tau = k_t I$$

k_t : torque constant (N·m/A)

Example: $k_t = 0.05$, $I = 2A \rightarrow \tau = 0.1 N \cdot m$

At steady state ($dI/dt \approx 0$):

$$\omega = \frac{V - IR}{k_e}$$

- Higher $V \rightarrow$ faster speed
- Larger load $\rightarrow$ higher $I \rightarrow$ lower speed

$$P_{elec} = VI, \quad P_{mech} = \tau\omega$$

$$\eta = \frac{P_{mech}}{P_{elec}} \times 100\%$$



PWM Speed Control

Pulse Width Modulation simulates analog voltage

$$V_{avg} = D \cdot V_s$$

D : duty cycle (0–1) V_s : supply voltage

Duty	Effective V (12 V supply)	Speed
25 %	3 V	Slow
50 %	6 V	Medium
75 %	9 V	Fast
100 %	12 V	Max

Direction Control (H-Bridge)

- Four transistors form an “H” configuration
- Enables **forward**, **reverse**, and **brake**
- Controlled by two PWM inputs (A/B)



Load current:

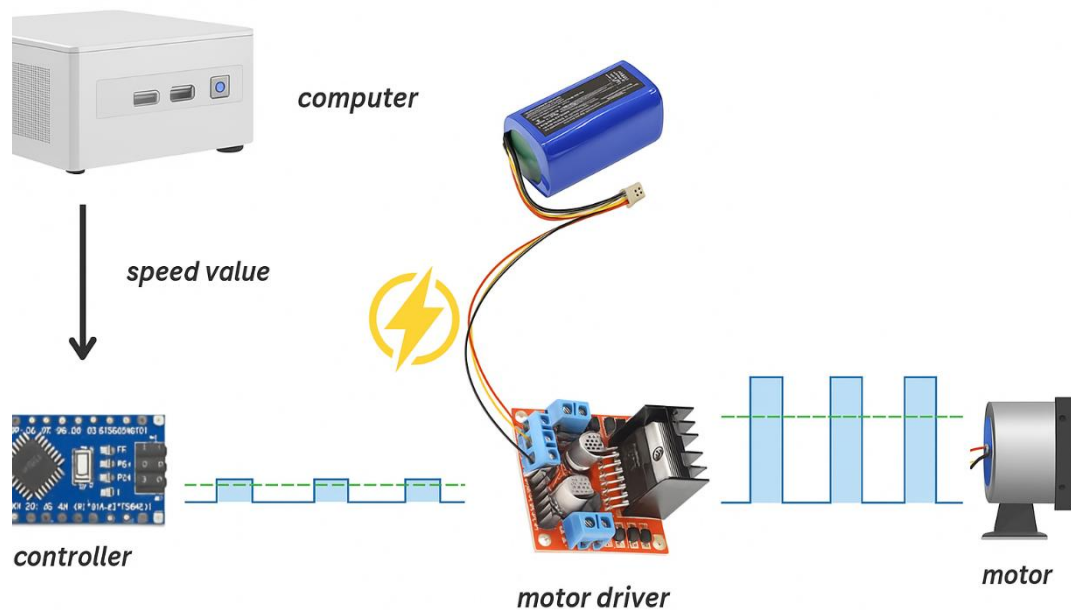
$$I = \frac{V_{supply} - V_{drop}}{R_{motor}}$$

System Integration

Signal flow:

Computer (ROS AI) → Motor Controller → Driver → Motor → Encoder → Controller

ROS Topics: `/cmd_vel` ↔ `/odom`

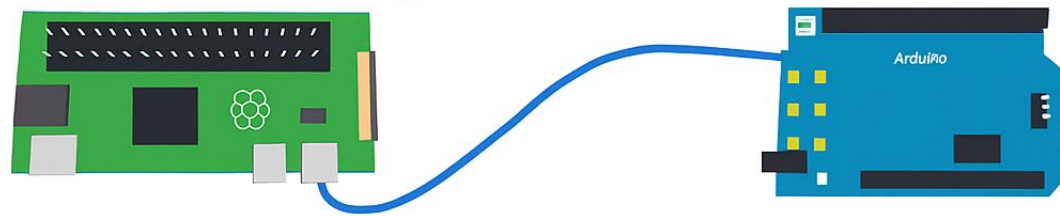


Serial Communication and Networking

- Demonstration: control robot motors **directly from your computer**
- Goal: understand how to **connect, program, and control** DC motors
- Required: Arduino Nano + L298N + Motors + Battery + Encoders
- Outcome: control motor speed via **PWM (Pulse Width Modulation)** from ROS 2/PC

Serial Communication - Application

Raspberry Pi + Arduino

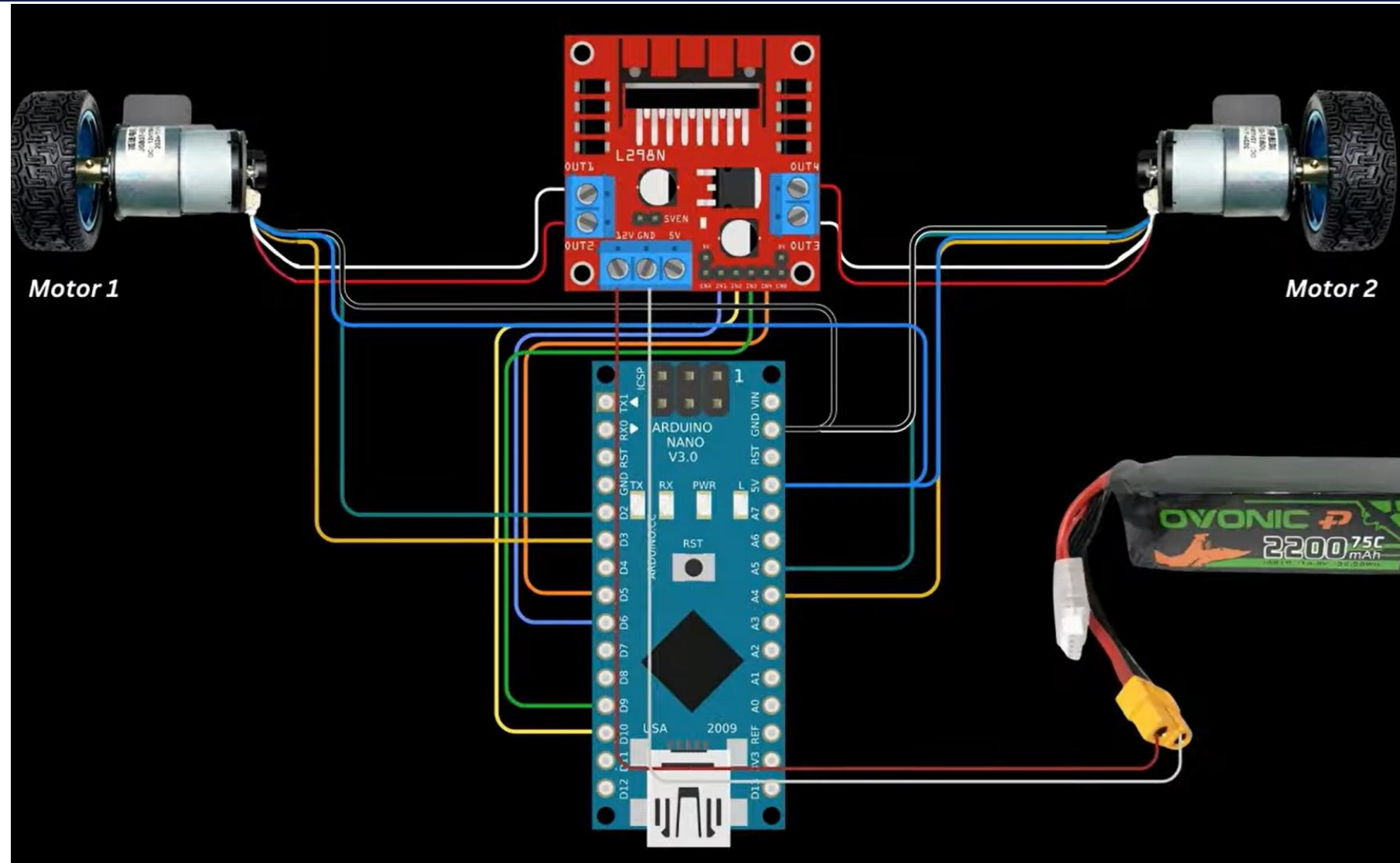


- Processing power
- WiFi
- USB controller
- Camera
- Web server
- Audio

- Accurate signal timing
- Real-time response
- 5V output voltage
- Analog inputs
- Libraries available

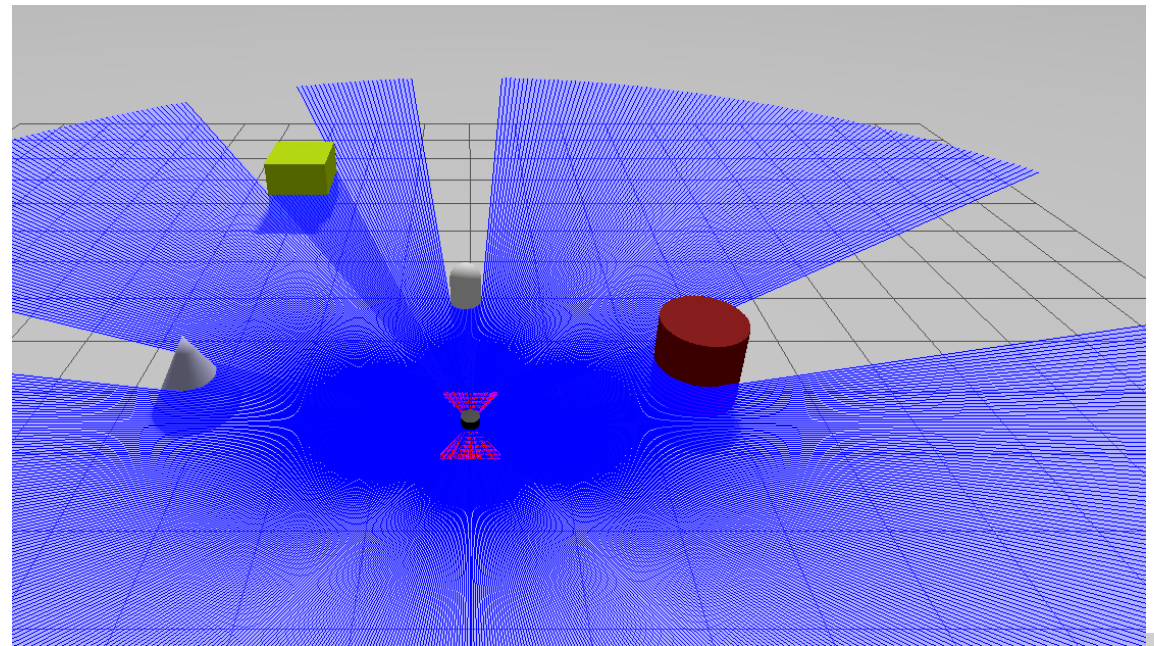
Share the load each doing
what they do best

System Overview



Activity: Develop a Model of a Mobile Robot with a Lidar Sensor

- **Objective:**
 - Learn how to create a differential robot model in Gazebo and integrate a Lidar sensor using ROS 2 Jazzy.
- **Key points:**
 - Build a mobile robot model in Gazebo
 - Attach a Lidar sensor to the robot
 - Launch the simulation in ROS 2 Jazzy
 - Read and visualize Lidar sensor data.



Activity: Setup and Package Creation Steps

```
source /opt/ros/jazzy/setup.bash
```

```
sudo apt-get update && sudo apt-get upgrade
```

```
cd ~
```

```
mkdir -p ~/lec10_ws/src
```

```
cd ~/lec10_ws/src
```

```
ros2 pkg create --build-type ament_python mobile_robot
```

```
cd ~/lec10_ws/
```

```
Code .
```

Activity: Setup and Package Creation Steps– continue

- ✓ **Create three folders: “launch”, “model”, and “parameters” under the package mobile_robot**
- ✓ **Under model folder, create “robot.xacro” file and paste the code provided on (BB - robot.xacro)**
- ✓ **Under model folder, create “robot.gazebo” file and paste the code provided on (BB - robot.gazebo)**
- ✓ **Under parameters folder, create “bridge_parameters.yaml” file and paste the code provided on (BB - bridge_parameters.yaml)**
- ✓ **Under launch folder, create “gazebo_model.launch.py” file and paste the code provided on (BB - gazebo_model.launch.py)**

Activity: Setup and Package Creation Steps– continue

- Open “package.xml” and add the following lines:

```
<depend>joint_state_publisher</depend>
```

```
<depend>robot_state_publisher</depend>
```

```
<exec_depend>xacro</exec_depend>
```

```
<depend>ros_gz</depend>
```

```
<depend>ros_gz_sim</depend>
```

```
<depend>ros_gz_bridge</depend>
```

Activity: Setup and Package Creation Steps– continue

- **Open “setup.py file” and add the following:**
 - **At the top of the file add:**

```
import os
```

```
from glob import glob
```

Activity: Setup and Package Creation Steps– continue

- Find the “data_files” list and modify it as shown below – only the highlighted lines need to be added.

```
data_files=[  
    ('share/ament_index/resource_index/packages',  
     ['resource/' + package_name]),  
    ('share/' + package_name, ['package.xml']),  
    (os.path.join('share', package_name, 'launch'), glob('launch/*')),  
    (os.path.join('share', package_name, 'model'), glob('model/*')),  
    (os.path.join('share', package_name, 'parameters'), glob('parameters/*')),  
    ],
```

Activity: Setup and Package Creation Steps– continue

- Back to CL

```
source /opt/ros/jazzy/setup.bash
```

```
cd ~/lec10_ws/
```

```
sudo apt-get install python3-rosdep
```

```
sudo rosdep init
```

```
rosdep update
```

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

```
colcon build
```

```
source ~/lec10_ws/install/setup.bash
```

```
ros2 launch mobile_robot gazebo_model.launch.py
```

Running the Gazebo Simulation

1. Start Simulation

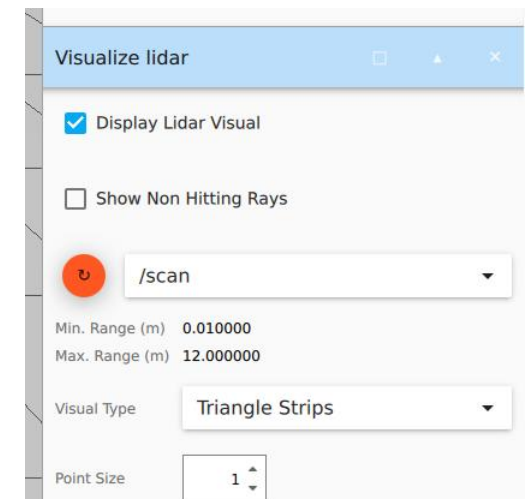
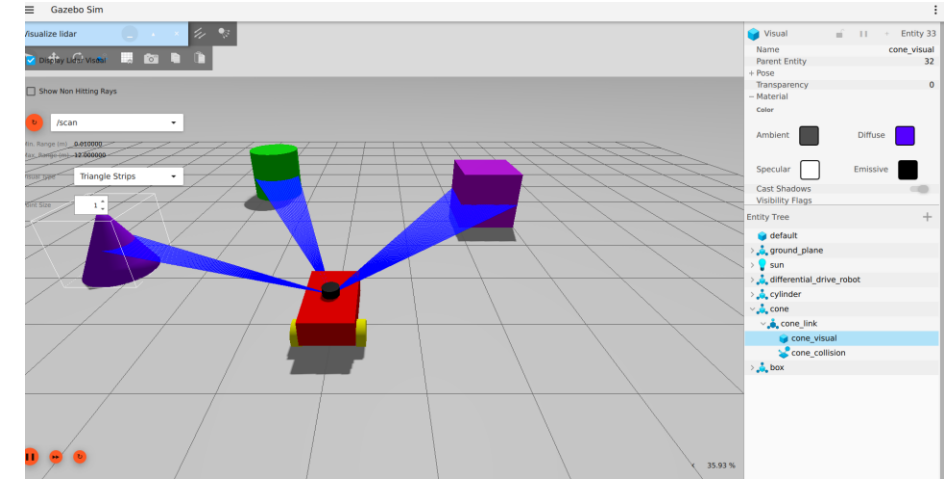
- Launch the Gazebo simulation.

2. Visualize Lidar Data

- Click the **three dots** in the upper-right corner.
- Search for “**Visualize Lidar**”.
- The *Visualize Lidar* menu will appear in the lower-right corner.
- Click the **orange button** (arrow and circle icon) to display lidar rays.
- (Optional) Turn off “**Show Non-Hitting Rays**” to display only reflected beams.

3. Add Objects to the Scene

- From the upper-left menu, select a **box** or other shape.
- Add objects to the scene for the lidar to detect.



Lidar data

- Open a new terminal and source it

```
source /opt/ros/jazzy/setup.bash
```

```
ros2 topic list
```

```
ros2 topic info /scan
```

```
ros2 interface show sensor_msgs/msg/LaserScan
```

```
ros2 topic echo /scan --field ranges
```

teleop node

- Open a new terminal and source it

```
source /opt/ros/jazzy/setup.bash
```

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

- Navigate the robot

Summary



- Integrated **LIDAR** and **motor systems** into the ROS 2 robot.
- Simulated a **2D LIDAR** in Gazebo and visualized scans in RViz.
- Connected a real **RPLIDAR A1** using the `rplidar_ros` driver.
- Wired and powered **DC motors**, **L298N driver**, **Arduino Nano**, and **encoders**.
- Flashed the **ROS-Arduino bridge** to control motor speed from the PC.
- Tested communication using **SerialTool** and verified encoder feedback.
- Robot now performs full sensor-actuator integration, ready for **motor calibration** next week.

THANK YOU.



WE ARE

HUMBER